RAPPORT DE PROJET

CONCEPTION ET DÉVELOPPEMENT D'UNE APPLICATION DE GESTION COMMERCIALE

VENTES • STOCKS • CLIENTS • TABLEAU DE BORD

Filière : Génie Informatique
Matière : Programmation WEB 3
Superviseur : TABA-CHABI Kassim

Présenté par :

MONTCHO Odilon
KPASSI Gildas
EDEGAN Belmonde
OGOUTCHORO Julie
TCHETCHE DOSSA Antoinette
ZANNOUN Karim

Année académique : 2026 – 2027

Document de présentation technique et fonctionnelle

AVANT-PROPOS

Le présent rapport décrit le travail de conception et de réalisation d’une application web de gestion commerciale consacrée au suivi des produits, des clients, des ventes et des mouvements de stock. Le projet a été développé dans le cadre de la matière Programmation WEB 3, en filière Génie Informatique. Il met en œuvre une architecture séparant un backend chargé des traitements et de l’accès aux données, et un frontend destiné à l’interaction avec les utilisateurs.

Le backend repose sur Laravel 12 et expose une API REST. Le frontend est réalisé avec React 19 et communique avec cette API au moyen d’Axios. Les données sont enregistrées dans une base MySQL ou MariaDB nommée commerciale_db.

Ce document présente le contexte, les objectifs, l’analyse fonctionnelle, l’architecture, les principales fonctionnalités, le modèle de données, les procédures d’installation et les vérifications techniques. Les éléments décrits correspondent aux spécifications communiquées pour le projet ; les résultats de tests et les captures d’écran peuvent être ajoutés à partir d’une exécution réelle de l’application.

SOMMAIRE

Rubrique	Page indicative
Introduction générale	3
1. Présentation générale du projet	4
2. Analyse des besoins et spécifications fonctionnelles	5
3. Architecture technique et environnement	8
4. Conception de la base de données	10
5. Réalisation du backend Laravel	12
6. Réalisation du frontend React	14
7. Installation et mise en service	16
8. Tests, sécurité et limites	18
Conclusion générale et perspectives	20
Annexes : API et commandes utiles	21

La pagination est générée automatiquement par le lecteur PDF ; les numéros ci-dessus constituent un repérage indicatif.

INTRODUCTION GÉNÉRALE

La gestion commerciale constitue une activité essentielle pour toute structure qui commercialise des biens. Elle implique le suivi des articles disponibles, la maîtrise des prix, l'enregistrement des ventes, la gestion des clients et le contrôle permanent des quantités en stock. Lorsque ces opérations sont effectuées manuellement ou au moyen de fichiers dispersés, les risques d'erreur, de perte d'informations, de rupture de stock et de difficultés de suivi augmentent.

Dans ce contexte, le groupe a entrepris la réalisation d'une application web centralisée permettant de gérer les principales opérations commerciales. Le système regroupe dans une même interface le catalogue des produits, les catégories, les clients, les ventes, les mouvements de stock et les indicateurs de pilotage.

Problématique

Comment concevoir une application web fiable et simple d'utilisation qui permette d'enregistrer les ventes, de contrôler automatiquement les quantités disponibles, d'assurer la traçabilité des mouvements de stock et de fournir une vue synthétique de l'activité commerciale ?

Objectif général

L'objectif général est de concevoir et développer une application web de gestion commerciale couvrant les opérations courantes de vente et de gestion de stock, avec une architecture moderne composée d'une API Laravel et d'une interface React.

Objectifs spécifiques

- Centraliser les informations relatives aux produits, catégories et clients.
- Permettre l'enregistrement, la consultation et l'annulation des ventes.
- Calculer automatiquement les montants, remises, taxes et totaux.
- Contrôler la disponibilité des produits avant la validation d'une vente.
- Tracer les entrées, sorties et corrections de stock.
- Signaler les produits en rupture ou sous leur seuil minimal.
- Présenter des indicateurs et graphiques utiles au suivi de l'activité.
- Mettre en pratique les technologies étudiées en programmation web.

Démarche adoptée

La démarche suivie s'articule autour de l'identification des besoins, de la définition des règles de gestion, de la conception des données, de la réalisation du backend, du développement de l'interface, puis de la vérification des fonctionnalités. Le découpage en deux parties facilite la séparation des responsabilités et l'évolution indépendante de l'API et de l'interface.

1. PRÉSENTATION GÉNÉRALE DU PROJET

1.1. Intitulé et nature du projet

Le projet, intitulé « Développement d'une application de gestion de ventes et des stocks », est une application web de gestion commerciale. Il est organisé en deux sous-projets : backend pour la logique métier et l'accès aux données, et frontend pour l'interface utilisateur.

1.2. Périmètre fonctionnel

Le périmètre couvre six grands domaines : le catalogue des produits, les catégories, les clients, les ventes, les mouvements de stock et le tableau de bord avec alertes. L'application est conçue pour être utilisée en français et s'adapter aux écrans d'ordinateur, de tablette et de téléphone.

1.3. Utilisateurs concernés

Dans un contexte d'utilisation commerciale, l'application peut être utilisée par un responsable de stock, un agent de vente, un gestionnaire de clientèle ou un responsable souhaitant consulter les statistiques. Pour les besoins du travail pratique, aucune authentification n'est exigée : l'interface est directement accessible. Cette décision simplifie la démonstration, mais ne convient pas à un déploiement réel sans mécanisme de contrôle d'accès.

1.4. Résultats attendus

- Une base de données structurée et cohérente.
- Une API REST permettant les opérations de consultation et de gestion.
- Une interface réactive facilitant les opérations quotidiennes.
- Un historique exploitable des ventes et des mouvements de stock.
- Des indicateurs permettant d'apprécier l'évolution de l'activité.

2. ANALYSE DES BESOINS ET SPÉCIFICATIONS FONCTIONNELLES

2.1. Gestion du catalogue des produits

Le module Produits permet de créer, consulter, modifier, supprimer ou désactiver les articles. Chaque produit comporte notamment un nom, une référence, une catégorie, une unité, un prix d'achat, un prix de vente, une marge calculée, une quantité en stock et un seuil d'alerte.

- La référence peut être générée automatiquement lorsqu'elle n'est pas fournie.
- Les listes peuvent être recherchées, filtrées par catégorie ou niveau de stock, puis triées.
- Les produits peuvent être activés ou désactivés.
- Le stock ne se modifie pas directement lors d'une modification ordinaire du produit : il évolue par des mouvements enregistrés afin de préserver la traçabilité.

2.2. Gestion des catégories

Les catégories servent à organiser le catalogue. Chaque catégorie possède un libellé et une couleur d'identification. L'interface peut également afficher le nombre de produits rattachés à chaque catégorie. La suppression d'une catégorie est refusée lorsqu'elle contient encore des produits, afin d'éviter de laisser des références incohérentes.

2.3. Gestion des clients

Le module Clients permet de constituer une fiche pour chaque client avec ses coordonnées, sa ville et des notes complémentaires. La recherche peut s'effectuer à partir du nom, de l'adresse électronique, du téléphone ou de la ville. La fiche détaillée présente les dix dernières ventes et le montant total dépensé, en excluant les ventes annulées. La suppression est bloquée lorsqu'un client possède déjà des ventes.

2.4. Gestion des ventes

L'encaissement suit plusieurs étapes : choix du client, constitution du panier, définition de la date, sélection du moyen de paiement, saisie éventuelle d'une remise, application du taux de TVA et ajout d'une note. Le serveur recalcule et vérifie les montants avant l'enregistrement.

Règles de calcul

Le sous-total correspond à la somme des montants des lignes de vente. La remise est déduite du sous-total. La TVA est calculée sur le montant après remise. Le total TTC est donc déterminé par la formule suivante :

Total TTC = (Sous-total − Remise) + TVA

Pour chaque ligne, le montant est obtenu en multipliant la quantité par le prix unitaire appliqué. Le serveur vérifie que la remise ne dépasse pas le sous-total et que le taux de TVA est compris entre 0 et 100 %. Les montants affichés dans l'application sont formatés selon les conventions françaises et en euros, conformément aux spécifications fournies.

Validation et annulation

- Le client et les produits référencés doivent exister.
- Une vente comporte au moins une ligne et chaque quantité doit être un entier strictement positif.
- Le produit doit être actif et son stock doit être suffisant.
- Une vente validée diminue les quantités disponibles.
- Lorsqu'une vente est annulée, son statut devient « Annulée » et les quantités vendues sont réintégrées au stock.

2.5. Gestion du stock

Le module Stock enregistre les entrées, les sorties et les corrections d'inventaire. Chaque mouvement est horodaté et conserve la quantité avant opération, la quantité après opération ainsi qu'un motif. Les sorties liées à une vente ou à son annulation peuvent être rattachées à la vente d'origine. Le système refuse toute sortie supérieure au stock disponible.

2.6. Tableau de bord et alertes

Le tableau de bord rassemble les indicateurs de gestion : chiffre d'affaires du jour, du mois et cumulé, nombre de ventes, panier moyen, évolution par rapport au mois précédent, valeur du stock, nombre de produits et de clients, ainsi que le nombre d'alertes.

- Graphique des ventes des sept derniers jours.
- Évolution du chiffre d'affaires sur douze mois.
- Répartition des ventes par catégorie et par moyen de paiement.
- Meilleurs produits du mois et ventes récentes.

- Liste des produits en rupture ou sous le seuil d’alerte.
- Badge de notification et raccourcis vers les produits concernés.

3. ARCHITECTURE TECHNIQUE ET ENVIRONNEMENT

3.1. Architecture générale

L'application adopte une architecture client–serveur séparée en deux parties. Le navigateur exécute le frontend React. Celui-ci envoie des requêtes HTTP à l'API REST développée avec Laravel. L'API applique les règles métier, dialogue avec MySQL ou MariaDB, puis retourne des réponses au format JSON.

Couche	Technologies	Responsabilité
Interface utilisateur	React 19, Vite, React Router 7, CSS	Affichage des pages, formulaires, navigation et graphiques.
Communication	Axios, API REST, JSON	Transmission des demandes entre le navigateur et le backend.
Backend	Laravel 12, PHP 8.2	Validation, règles métier, calculs, transactions et réponses JSON.
Persistance	MySQL / MariaDB	Stockage durable des produits, clients, ventes et mouvements.

3.2. Technologies utilisées

Laravel 12 et PHP 8.2

Laravel est le framework PHP utilisé pour construire le backend. Il fournit notamment le routage, les contrôleurs, les modèles, les migrations, les validateurs et les mécanismes de gestion des transactions. PHP 8.2 constitue l'environnement d'exécution indiqué pour le projet.

React 19 et Vite

React permet de construire l'interface sous forme de composants réutilisables. Vite assure le serveur de développement et la génération de la version de production. React Router 7 gère la navigation entre les pages.

Axios et Recharts

Axios est utilisé pour envoyer les requêtes HTTP vers l'API et récupérer les données. Recharts permet de représenter visuellement les indicateurs statistiques au moyen de graphiques.

MySQL / MariaDB

Le système de gestion de base de données conserve les informations de l'application dans la base commerciale_db. Le jeu de caractères utf8mb4 permet notamment de stocker correctement les caractères accentués.

3.3. Communication entre les parties

En environnement local, Laravel expose l'API sur http://127.0.0.1:8000 et Vite sert l'interface sur http://127.0.0.1:5173. Le fichier vite.config.js configure un proxy qui redirige les requêtes commençant par /api vers Laravel. Cette organisation évite de devoir définir une URL distante pour chaque appel durant le développement.

3.4. Organisation des dossiers

Répertoire	Contenu principal
backend/app/Http/Controllers	Contrôleurs du tableau de bord, produits, catégories, clients, ventes et stock.
backend/app/Http/Requests	Validation des données reçues.
backend/app/Http/Resources	Mise en forme des réponses JSON.
backend/app/Models	Modèles représentant les entités de la base.
backend/app/Services	Services spécialisés, notamment SaleService et StockService.
backend/database	Migrations et seeders.
backend/routes/api.php	Déclaration des routes de l'API REST.
backend/tests/Feature	Tests fonctionnels du backend.
frontend/src/api	Configuration et appels à l'API.
frontend/src/components	Composants réutilisables : modales, formulaires, tableaux et navigation.
frontend/src/pages	Pages du tableau de bord, produits, catégories, clients, ventes et stock.
frontend/src/styles et utils	Styles CSS, formats, libellés et fonctions utilitaires.

4. CONCEPTION DE LA BASE DE DONNÉES

4.1. Présentation des entités

La base de données relationnelle est organisée autour des principales entités métier. Les tables permettent de distinguer les données de référence (catégories, produits et clients), les opérations commerciales (ventes et lignes de vente) et la traçabilité (mouvements de stock).

Table	Rôle
categories	Enregistre les catégories auxquelles les produits peuvent appartenir.
products	Contient les informations descriptives, les prix, l'état et la quantité disponible des produits.
clients	Conserve les coordonnées et les informations complémentaires des clients.
sales	Enregistre l'en-tête d'une vente : client, date, paiement, totaux et statut.
sale_items	Détaille les produits, quantités et prix unitaires de chaque vente.
stock_movements	Journalise les entrées, sorties et corrections de stock.

4.2. Relations entre les tables

Les principales associations sont les suivantes :

- Une catégorie peut regrouper plusieurs produits ; chaque produit est rattaché à une catégorie.
- Un client peut effectuer plusieurs ventes ; chaque vente est associée à un client.
- Une vente contient une ou plusieurs lignes de vente ; chaque ligne désigne un produit.
- Un produit peut être concerné par plusieurs mouvements de stock.
- Un mouvement peut être lié à une vente d'origine lorsque le mouvement découle d'une vente ou de son annulation.

Schéma relationnel simplifié :

Relation	Cardinalité
categories → products	1 à plusieurs (1,n)
clients → sales	1 à plusieurs (1,n)
sales → sale_items	1 à plusieurs (1,n)
products → sale_items	1 à plusieurs (1,n)
products → stock_movements	1 à plusieurs (1,n)
sales → stock_movements	0 ou 1 vente associée à un mouvement, selon le cas

4.3. Contraintes d'intégrité

- La quantité products.stock est un entier non signé : elle ne doit pas devenir négative.
- Le statut d'une vente prend les valeurs completed ou cancelled.
- Les lignes de vente conservent le nom et la référence du produit au moment de l'opération afin de préserver l'historique, même si le catalogue évolue ensuite.

- Les champs stock_before et stock_after permettent de reconstituer l’évolution du stock.
- Les suppressions sont conditionnées par l’existence de dépendances : un produit ou un client déjà utilisé dans une vente ne peut pas être supprimé ; une catégorie non vide ne peut pas être supprimée.

4.4. Intérêt de la traçabilité

La conservation des mouvements permet de comprendre pourquoi une quantité a changé, à quel moment et dans quel contexte. Elle facilite les contrôles, la recherche d’erreurs et la justification des écarts d’inventaire. Elle évite également qu’une modification manuelle du champ de stock efface l’historique des opérations.

5. RÉALISATION DU BACKEND LARAVEL

5.1. Rôle du backend

Le backend constitue le cœur fonctionnel de l'application. Il reçoit les requêtes du frontend, vérifie les données, applique les règles métier, exécute les opérations en base de données et retourne des réponses JSON. Les routes sont regroupées sous le préfixe /api.

5.2. Organisation en couches

- Routes : associent une méthode HTTP et une URL à une action.
- Contrôleurs : reçoivent les requêtes et coordonnent les traitements.
- Form Requests : centralisent les règles de validation des entrées.
- Modèles : représentent les tables et leurs relations.
- Resources : structurent les réponses JSON.
- Services : isolent les traitements métier complexes, notamment la vente et le stock.

5.3. Principales ressources de l'API

Domaine	Méthodes et routes principales	Fonctions
Santé et tableau de bord	GET /api/health GET /api/dashboard	État de l'API, connexion à la base, indicateurs et alertes.
Produits	GET, POST /api/products GET, PUT/PATCH, DELETE /api/products/{id}	Liste, création, consultation, modification et suppression conditionnelle.
Stock produit	POST /api/products/{id}/stock-adjustments GET /api/products/{id}/movements	Ajustement et historique des mouvements.
Catégories	GET, POST /api/categories GET, PUT/PATCH, DELETE /api/categories/{id}	Gestion des catégories.
Clients	GET, POST /api/clients GET, PUT/PATCH, DELETE /api/clients/{id}	Gestion des fiches clients et de leur historique.
Ventes	GET, POST /api/sales GET /api/sales/{id} POST /api/sales/{id}/cancel	Historique, enregistrement, détail et annulation.
Mouvements	GET /api/stock-movements	Journal paginé et filtrable des mouvements.

5.4. Traitement d'une vente

1. Le frontend transmet le client, la date, le moyen de paiement, les paramètres fiscaux et les lignes du panier.
2. Le backend vérifie l'existence du client et de chaque produit, l'activité des produits et la validité des quantités.
3. Le serveur vérifie la disponibilité du stock et les règles relatives à la remise et à la TVA.
4. Les montants sont calculés côté serveur afin de ne pas dépendre uniquement des valeurs affichées dans le navigateur.

5 La vente, ses lignes et les mouvements de stock correspondants sont enregistrés de manière cohérente.

6 Une réponse JSON est renvoyée au frontend pour confirmer l’opération ou signaler une erreur.

5.5. Transactions et concurrence

La création et l’annulation des ventes sont exécutées dans une transaction SQL. Le verrouillage pessimiste au moyen de lockForUpdate permet de verrouiller les lignes concernées pendant le traitement. Cette approche réduit le risque que deux opérations concurrentes vendent simultanément une quantité qui n’est disponible qu’une seule fois. En cas d’échec, la transaction peut être annulée afin d’éviter un enregistrement partiel.

5.6. Services métier

SaleService prend en charge les opérations liées aux ventes, notamment les contrôles et les calculs. StockService centralise les règles d’entrée, de sortie et de correction de stock. Cette séparation facilite la maintenance du code et limite la duplication des règles.

6. RÉALISATION DU FRONTEND REACT

6.1. Rôle de l’interface

Le frontend constitue la partie visible de l’application. Il permet à l’utilisateur de naviguer entre les modules, de consulter les informations, de remplir des formulaires et de visualiser les résultats renvoyés par l’API. L’interface est rédigée en français et conçue pour s’adapter aux écrans de bureau, de tablette et de téléphone.

6.2. Pages principales

Page	Contenu et opérations
Tableau de bord	Indicateurs de chiffre d’affaires, ventes, panier moyen, valeur du stock, graphiques et alertes.
Produits	Catalogue, recherche, filtres, création, modification, désactivation et accès aux mouvements.
Catégories	Liste, création, modification, couleur et nombre de produits associés.
Clients	Fiches, coordonnées, recherche, historique et total dépensé.
Ventes	Panier, encaissement, historique filtrable, détail et annulation.
Stock	Entrées, sorties, corrections d’inventaire et journal des mouvements.

6.3. Composants réutilisables

Les composants communs, tels que les modales, formulaires, tableaux, éléments de navigation et notifications toast, permettent d’uniformiser l’interface. Les pages sont ainsi plus faciles à maintenir et les comportements communs peuvent être corrigés à un seul endroit.

6.4. Communication avec l’API

Axios est utilisé pour centraliser les appels HTTP. Le frontend envoie des requêtes vers les routes /api et traite les réponses JSON. En cas de validation refusée ou d’erreur serveur, les messages peuvent être présentés à l’utilisateur au moyen de notifications ou d’indications dans les formulaires.

6.5. Présentation des données

Les fonctions utilitaires assurent le formatage des dates, des montants et des libellés de statut. Recharts est utilisé pour présenter les tendances et répartitions sous forme de graphiques. Le CSS personnalisé définit le système visuel et les adaptations nécessaires aux différentes tailles d’écran.

6.6. Parcours type d’un utilisateur

1. Depuis le tableau de bord, l’utilisateur consulte les indicateurs et les alertes de stock.
2. Il ouvre le catalogue pour rechercher un produit ou vérifier sa disponibilité.
3. Il sélectionne un client et ajoute les articles souhaités au panier.
4. Il renseigne les paramètres de paiement, de remise et de TVA.
5. Il valide la vente ; le serveur contrôle le stock et enregistre l’opération.
6. Il peut ensuite consulter la vente dans l’historique et retrouver les mouvements correspondants.

7. INSTALLATION ET MISE EN SERVICE

7.1. Prérequis

- PHP 8.2 ou une version supérieure, avec les extensions pdo_mysql, mbstring, openssl et fileinfo.
- Composer pour la gestion des dépendances PHP.
- Node.js 20 ou une version supérieure et npm.
- MySQL ou MariaDB, par exemple via XAMPP, WAMP ou MAMP.
- Un terminal et un éditeur de code pour exécuter et configurer le projet.

7.2. Création de la base de données

Après le démarrage du serveur MySQL, créer la base avec la commande suivante :

```
CREATE DATABASE commerciale_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Sous Windows avec XAMPP, la commande peut être exécutée depuis un terminal :

```
C:\xampp\mysql\bin\mysql.exe -u root -e CREATE DATABASE IF NOT EXISTS commerciale_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

7.3. Installation du backend

Depuis le dossier backend, installer les dépendances, préparer le fichier d'environnement, générer la clé et créer les tables avec les données de démonstration :

```
cd backend
composer install
copy .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan serve
```

Le fichier .env.example est configuré pour une base locale. Les paramètres DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME et DB_PASSWORD doivent être adaptés si l'installation MySQL utilise d'autres valeurs.

7.4. Importation alternative du fichier SQL

Le projet peut également fournir un fichier commerciale_db.sql à la racine. Celui-ci peut être importé dans une base vide. Dans ce cas, il ne faut pas exécuter ensuite migrate --seed, puisque le dump contient déjà le schéma et les données.

```
C:\xampp\mysql\bin\mysql.exe -u root commerciale_db < commerciale_db.sql
```

7.5. Installation du frontend

Dans un second terminal, depuis le dossier frontend :

```
cd frontend
npm install
npm run dev
```

L'interface est alors accessible à l'adresse http://127.0.0.1:5173. L'API Laravel est accessible à l'adresse http://127.0.0.1:8000. Le proxy configuré dans Vite redirige les requêtes /api vers le backend.

7.6. Données de démonstration

Le seeder fournit un jeu de données destiné à faciliter les essais :

- 5 catégories et 20 produits, dont certains en rupture ou en stock faible ;
- 10 clients ;
- 25 ventes, dont 24 validées et 1 annulée, réparties sur les derniers mois ;
- 80 mouvements de stock.

Pour réinitialiser complètement les données de démonstration, la commande prévue est :

```
php artisan migrate:fresh --seed
```

Attention : cette commande supprime les tables existantes avant de les recréer. Elle doit donc être utilisée uniquement sur une base de développement dont les données peuvent être effacées.

8. TESTS, SÉCURITÉ ET LIMITES

8.1. Vérifications du backend

Les tests fonctionnels sont lancés depuis le dossier backend avec la commande :

```
php artisan test
```

Les scénarios prévus couvrent notamment :

- Les opérations CRUD des catégories, clients et produits.
- La génération automatique des références et le mouvement de stock initial.
- Les filtres de produits en alerte, en stock faible ou en rupture.
- Les ajustements de stock et le refus d’un stock négatif.
- Les calculs du sous-total, de la remise, de la TVA et du total de vente.
- Le refus d’une vente avec un stock insuffisant ou un produit inactif.
- L’annulation d’une vente et la restauration des quantités.
- L’exclusion des ventes annulées des statistiques.
- Les routes du tableau de bord.

8.2. Vérifications du frontend

Les commandes prévues pour contrôler la qualité du frontend sont :

```
npm run lint
npm run build
```

La première commande recherche les problèmes de qualité ou de syntaxe détectables par l’outil de lint. La seconde vérifie que l’application peut être compilée pour la production. Les résultats effectifs doivent être consignés après exécution dans l’environnement du groupe.

8.3. Sécurité et intégrité

Plusieurs règles contribuent à la fiabilité des opérations : validation des données côté serveur, contrôle du stock disponible, transactions SQL, verrouillage pessimiste, interdiction des sorties supérieures au stock et refus de certaines suppressions lorsque des données sont liées.

Toutefois, l’application est volontairement dépourvue d’authentification dans le cadre de ce travail pratique. Elle ne doit pas être exposée telle quelle à un réseau public ou utilisée pour gérer une activité réelle. Une version de production devrait notamment intégrer Laravel Sanctum, une gestion des rôles et permissions, la protection des routes, une politique de sauvegarde et une configuration HTTPS.

8.4. Limites identifiées

- Absence de comptes utilisateurs et de journal d’audit par utilisateur.
- Absence de rôles distincts pour les vendeurs, gestionnaires et administrateurs.
- Application destinée à une démonstration locale selon la configuration décrite.
- Les montants sont affichés en euros dans la version spécifiée ; une adaptation de devise et de règles fiscales peut être nécessaire selon le contexte d’exploitation.

- Les performances et la sécurité en production doivent être évaluées dans un environnement d’hébergement réel.

8.5. Perspectives d’amélioration

- Ajouter l’authentification, les rôles et les permissions avec Laravel Sanctum.
- Mettre en place une gestion des fournisseurs et des commandes d’approvisionnement.
- Ajouter l’impression ou l’export PDF des factures et reçus.
- Prévoir l’export Excel ou CSV des ventes, clients et mouvements.
- Ajouter des sauvegardes automatiques et une procédure de restauration.
- Permettre la configuration de la devise, de la TVA et des seuils par catégorie ou produit.
- Déployer l’application sur un serveur sécurisé avec journalisation et supervision.

CONCLUSION GÉNÉRALE

Le projet de gestion commerciale, ventes et stocks a pour ambition de réunir dans une application web les opérations essentielles d'une activité de vente. La séparation entre le backend Laravel et le frontend React permet de distinguer clairement les traitements métier de la présentation des informations. L'API REST assure la communication entre les deux parties, tandis que MySQL ou MariaDB conserve les données.

Les fonctionnalités prévues couvrent la gestion du catalogue, des catégories et des clients, l'enregistrement et l'annulation des ventes, la traçabilité des mouvements de stock, les alertes de réapprovisionnement et le suivi statistique de l'activité. Les transactions SQL et les contrôles côté serveur contribuent à préserver la cohérence des opérations, en particulier lorsqu'une vente modifie les quantités disponibles.

Sur le plan pédagogique, ce travail permet de mobiliser plusieurs compétences : conception d'une base relationnelle, développement d'une API REST, programmation côté serveur avec Laravel, construction d'une interface avec React, communication HTTP, validation des données et tests. Il constitue une base évolutive qui pourrait être renforcée par l'authentification, les rôles, la facturation, les sauvegardes et le déploiement sécurisé.

En conclusion, cette réalisation répond au besoin pédagogique de concevoir une solution intégrée de gestion commerciale et offre une base technique pouvant être enrichie pour répondre à des exigences professionnelles plus avancées.

ANNEXE A — EXEMPLE DE REQUÊTE DE CRÉATION D'UNE VENTE

L'exemple ci-dessous illustre la structure JSON attendue par l'API lors de l'enregistrement d'une vente. Les identifiants doivent correspondre à des clients et produits existants dans la base.

```
{
 "client_id": 1,
 "sale_date": "2026-10-29T14:30:00",
 "payment_method": "card",
 "tax_rate": 20,
 "discount_amount": 5,
 "notes": "Livraison offerte",
 "items": [
  { "product_id": 1, "quantity": 2, "unit_price": 199.00 }
 ]
}
```

ANNEXE B — PRINCIPALES COMMANDES

Opération	Commande
Installer les dépendances PHP	composer install
Générer la clé Laravel	php artisan key:generate
Créer les tables et les données de démonstration	php artisan migrate --seed
Démarrer le serveur Laravel	php artisan serve
Installer les dépendances React	npm install
Démarrer Vite	npm run dev
Exécuter les tests backend	php artisan test
Vérifier le frontend	npm run lint
Compiler le frontend	npm run build
Réinitialiser la base de démonstration	php artisan migrate:fresh --seed

ANNEXE C — COMPOSITION DU GROUPE

N°	Nom et prénom
1	MONTCHO Odilon
2	KPASSI Gildas
3	EDEGAN Belmonde
4	OGOUTCHORO Julie
5	TCHETCHE DOSSA Antoinette
6	ZANNOUN Karim

Supervision : TABA-CHABI Kassim

Filière : Génie Informatique — Matière : Programmation WEB 3

Fin du rapport.